

CGA: Conditioning as Group Action

Richard Bao

richardbao419@gmail.com

Abstract

Networks are usually conditioned by scaling features (FiLM) or by generating weights (hypernetworks). Both learn what each condition does. Neither learns what two conditions do *together* unless training data shows them together. We condition differently: rotate the features, and make the rotation angles a linear function of the condition. Rotations add, so combined conditions produce combined effects automatically. The network handles condition combinations it never saw. We call the framework **CGA** (Conditioning as Group Action): the condition enters the operator linearly in an abelian Lie algebra, $T(c) = P \exp(A(Wc))P^\top$, so $T(c_1+c_2) = T(c_1)T(c_2)$ holds exactly for any weights. Known methods drop out as special cases: a diagonal algebra gives a compositional variant of FiLM, and the rotation algebra with scalar position gives RoPE. Our version costs $1.11\times$ FiLM’s conditioning compute and uses fewer parameters than every baseline. We test it under a pre-registered protocol: success margins, statistical tests, and data splits were fixed and committed before each run. Across five transformation suites (synthetic operators, dSprites, 3D Shapes), CGA has the lowest error on never-trained condition combinations, with complete seed separation ($p=9.1\times 10^{-5}$, $N=10$). It extrapolates to longer combinations than training ever contained. Replacing its linear map with an MLP, holding everything else fixed, raises error by up to 4.3×10^4 . The most expressive baselines generalize worst. Three pre-registered gates also returned negatives, and we report them: a $1.46\times$ fit penalty on rich scenes; a $7\times$ failure when the condition specifies image content (a rotation moves information and cannot add it); and no advantage in world-model rollouts, where the bottleneck is representation consistency, not operator composition. The negatives tell practitioners exactly where the method stops working. They also specify a successor, which we built and tested under two further pre-registered gates: **Complex FiLM** treats each feature pair as a complex number, with an expressive magnitude (FiLM’s content channel) and a linear phase (our composition channel). It beats FiLM by 36% on unseen change combinations and matches FiLM on content generation ($0.97\times$ its error) in the same diffusion setting where the pure rotation failed. One operator now covers both roles at FiLM-class cost.

1 The framework

Let $x \in \mathbb{R}^d$ be features and $c \in \mathbb{R}^k$ a condition. FiLM [1] computes $y = \gamma(c) \odot x + \beta(c)$. AdaLN in DiT blocks [2] is its normalized form. Hypernetworks emit full weights $W(c)$ [3]. All of these are instances of

$$y = T(c)x + \beta(c), \tag{1}$$

and they differ only in where the per-sample operator $T(c)$ comes from.

Definition 1 (Compositional conditioner). $T : \mathbb{R}^k \rightarrow \text{GL}(d)$ is compositional if it is a homomorphism from $(\mathbb{R}^k, +)$: $T(c_1+c_2) = T(c_1)T(c_2)$ and $T(0) = I$.

Proposition 1 (Exact composition by construction). Let $\mathfrak{a}$ be an abelian subalgebra of $\mathfrak{gl}(d)$, $A : \mathbb{R}^m \rightarrow \mathfrak{a}$ linear, $W \in \mathbb{R}^{m \times k}$, and P orthogonal. Then $T(c) = P \exp(A(Wc))P^\top$ is a compositional conditioner for every W, P . If $\mathfrak{a}$ is skew-symmetric, $T(c)$ is orthogonal.

Proof. $\exp(X)\exp(Y) = \exp(X+Y)$ for commuting X, Y ; $A \circ W$ is linear; conjugation preserves products; $\exp(0) = I$. $\square$

The point is simple. Composition is usually a behavior the network must learn from data. Here it is a property of the parametrization. It holds at initialization, during training, and far outside the training distribution. Training only decides *which* directions the condition rotates and *how fast*.

Special cases. Choices of $(\mathfrak{a}, c, P)$ recover deployed mechanisms:

$\mathfrak{a}$	condition	P	recovered mechanism
$\text{diag}(\mathbb{R}^d)$	c	I	<i>exponential FiLM</i> : $y = e^{Wc} \odot x$ (compositional)
$\mathfrak{so}(2)^{d/2}$	$c = n$ (scalar position)	I	RoPE [4]
$\mathfrak{so}(2)^{d/2}$	$c = n$, learned planes	learned	multiplicative GRAPE [5]
$\mathfrak{so}(2)^{d/2}$	general $c \in \mathbb{R}^k$	structured	this work (FiLM’s role)
$\text{diag} \times \mathfrak{so}(2)^{d/2}$	general c	canonical	Complex FiLM (the successor; §2)

Standard FiLM reaches the same diagonal operators through an MLP head $\gamma(c)$. The operators match; the composition law is missing. Section 2 shows the missing law is exactly what fails on unseen combinations: an ablation with the same operators, the same basis, and more compute loses by up to 4.3×10^4 once its map from condition to operator is an MLP. The PEFT literature (LoRA, OFT, BOFT, HRA, Group-and-Shuffle [6–10]) uses these operator families to fine-tune *weights*, one fixed transform per task. CGA uses them in FiLM’s role: a new operator per sample, applied to activations, where the composition question lives.

Cost. With $\mathfrak{a} = \mathfrak{so}(2)^{d/2}$, $\exp$ is a closed-form planar rotation: $3d$ FLOPs, no matrix exponential. The basis P is the whole budget question. A dense learned P costs $4d^2$ FLOPs per sample, which is $1.52\times$ FiLM’s entire conditioning path at $d=128$. Two fixes work. First, a structured orthogonal P (five block-diagonal Cayley layers with fixed shuffles, following Group-and-Shuffle) brings the total to $1.11\times$ FiLM. This P has 4,800 parameters against the 8,128 dimensions of $\text{SO}(128)$, and that is enough: it only needs to find the planes the condition acts on. Second, inside a transformer no explicit P is needed at all. The dense projections next to the rotation learn the basis, which is how RoPE works. The operator equals the identity at initialization and at $c=0$, and every singular value is 1. Unit tests pin all three properties.

2 Experiments

Protocol. Every suite is pre-registered. We fixed the success margins, the tests ($N=10$ seeds, one-sided Mann–Whitney $p \leq 0.01$, Cliff’s $|\delta| \geq 0.474$), and the data splits before each run, and committed them to version control. Each test split is read once. All arms share one backbone and one training recipe. A shared counter enforces the budget: our conditioning path stays within $1.20\times$ FiLM’s FLOPs and $1.05\times$ the smallest hypernetwork baseline’s parameters. Baselines may be bigger, and reach $48\times$ our parameter count. We report every verdict, including the negatives. Baselines: FiLM, conditional LayerNorm, Concat-MLP, a dense hypernetwork $I+\Delta W(c)$, a dynamic low-rank map $I+A(c)B(c)^\top$, and an MLP-head ablation of our own method.

Suites. **S1/S2 (synthetic):** learn $y = M(c)x$, where c selects a combination of 8 rotation primitives. S2 hides the primitives behind a random basis and also tests all 56 never-trained *triples*. **S3/S3b (dSprites) and S4 (3D Shapes):** image transformation. Encode a real image, apply $T(\Delta)$ to the latent for a factor change Δ , decode, and score pixel error against the true transformed image. OOD means never-trained *types* of two-factor changes. S3b and S4 add a categorical shape factor. **S5 (content):** a small diffusion transformer generates dSprites images from full factor specifications; held-out combinations have unique ground-truth images, so generation error is objective. **S6 (rollouts):** a world model applies the conditioning arm recurrently, one action per step; we test unseen action-type pairs and rollouts of length 10 and 20 after training on length 3. **S7 (the successor, two gates):** Complex FiLM, magnitude from FiLM’s MLP head and phase linear in the condition, rerun through the S3 transformation suite and the S5 content suite.

Findings. **(1) When conditions form a group, composition is solved.** On S2, CGA reaches 3.3×10^{-8} error on unseen pairs and 5.3×10^{-8} on never-trained triples. The best baseline sits five orders of magnitude higher, and every baseline gets worse as combinations get longer. CGA does not. Its learned angles match the true generators to 2×10^{-4} rad, so the operator is directly readable. **(2) The advantage survives learned representations.** On dSprites, every arm fits the training distribution equally well, so the comparison isolates composition. CGA’s error grows $1.25\times$ from training to unseen combinations ($1.63\times$ with the categorical factor). Every baseline grows $2.1\text{--}3.3\times$. CGA beats the strongest baseline by 53.8% (42.9%) at $39\times$ fewer conditioning FLOPs. **(3) The linear map is the mechanism.** The ablation keeps the basis and the operator family, spends more compute, and swaps only the linear map for an MLP. It loses by $4.3\times 10^4\times$ on synthetic triples and $1.55\times$ on dSprites. **(4) More capacity makes it worse.** The two

Table 1: Error on never-trained condition combinations (mean over 10 seeds). CGA is lowest in every column, with complete seed separation against the strongest hypernetwork baseline ($p=9.1\times 10^{-5}$, $\delta=-1.0$). S1–S3b pass the full pre-registered gate; S4 fails a separate fit criterion (text).

Conditioner	S1	S2	S3	S3b	S4	S7
FiLM	1.3×10^{-2}	1.6×10^{-2}	3.0×10^{-3}	5.9×10^{-3}	4.5×10^{-3}	3.1×10^{-3}
Concat-MLP	1.5×10^{-2}	1.1×10^{-2}	2.9×10^{-3}	6.3×10^{-3}	5.4×10^{-3}	–
Cond-LN	2.4×10^{-2}	2.7×10^{-2}	3.2×10^{-3}	6.1×10^{-3}	8.6×10^{-3}	–
Hypernetwork	1.9×10^{-3}	3.4×10^{-3}	3.8×10^{-3}	7.0×10^{-3}	9.3×10^{-3}	3.8×10^{-3}
Dyn. low-rank	5.7×10^{-3}	6.1×10^{-3}	5.1×10^{-3}	8.1×10^{-3}	2.0×10^{-2}	–
MLP-head (abl.)	–	1.1×10^{-3}	2.7×10^{-3}	5.0×10^{-3}	3.3×10^{-3}	–
Complex FiLM (lin.)	–	–	–	–	–	2.0×10^{-3}
Complex FiLM	–	–	–	–	–	2.0×10^{-3}
Lie (ours)	7.4×10^{-4}	3.3×10^{-8}	1.8×10^{-3}	4.0×10^{-3}	1.2×10^{-3}	1.8×10^{-3}

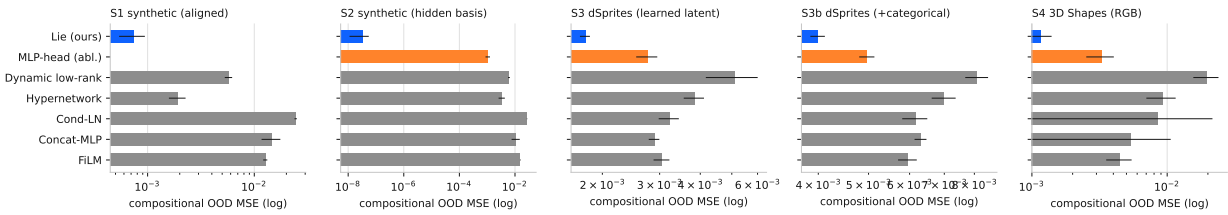


Figure 1: Error on unseen condition combinations, by conditioner (log scale; blue = CGA, orange = MLP-head ablation).

hypernetwork arms can express any operator. They post the worst generalization gaps on images (2.7–26 $\times$). Their capacity goes into memorizing the training combinations. **(5) Three registered negatives.** On 3D Shapes, CGA has the best OOD numbers (87.5% margin) but fits the training distribution 1.46 $\times$ worse than the best memorizer, which trips our no-regression gate. In the content suite, rotation conditioning fails (0.31 vs. film’s 0.04), and the ablation fails identically: a rotation moves information around and cannot add any, so it cannot carry “generate this image.” In the rollout suite, every arm compounds error at the same rate, and the mildly contractive low-rank arm degrades least. Each step of a rollout adds a small mismatch between the predicted and true latent. A contraction shrinks that mismatch at every step; an isometry carries it forever. Exact operator composition cannot repair an inconsistent representation.

3 Practical guidance: when to use this

Three rules for practitioners, from the eight suites.

Use phase conditioning when your conditions are changes that combine. Edit deltas, attribute offsets, pose or camera adjustments, applied once. The cost matches FiLM (1.11 $\times$ FLOPs, fewer parameters), training-distribution quality matches, and behavior on never-trained combinations is 43% to $10^5\times$ better. FiLM’s failure on new combinations is silent: nothing in training warns you. The guarantee removes that failure class.

For conditions that say what to generate, use Complex FiLM or keep FiLM. Pure rotation conditioning fails there (7 $\times$ worse; our registered negative). Complex FiLM matches FiLM on content (0.97 $\times$) while keeping the composition wins, so it is the candidate drop-in at our evidence scale; plain FiLM remains the conservative choice.

In rollouts, buy consistency, not composition. All conditioners compounded error at the same rate in our world-model suite. The bottleneck is whether the latent stays decodable, and a slightly contractive transition handles that better than an exact isometry.

All evidence comes from controlled 64 $\times$ 64 benchmarks with known factors. Nothing here is validated at production scale. The transformer variant costs less than FiLM’s marginal class path, so where the first rule applies, trying the swap is cheap.

4 Discussion

CGA does for conditioning what RoPE did for position: it moves composition from a behavior the network must learn into a law the parametrization guarantees. Four boundaries are measured. Categorical factors shrink the advantage. Rich scenes make orthogonality pay a fit penalty. Content specification defeats a transformation channel. Rollouts are limited by representation consistency, which no operator guarantee repairs. The first three pointed at one successor design, the algebra torus $\times$ diagonal, and S7 validated it: Complex FiLM carries content with its magnitude and composition with its phase. The fourth boundary calls for a consistency objective on the representation, which we are testing next. Related work has learned group actions on latents for world models [11, 12]; CGA differs by living in FiLM’s role under a matched budget, with composition holding by construction rather than by training objective.

Reproducibility. Every suite is pre-registered, and every number in this paper regenerates from committed logs. Unit tests pin the operator invariants and the budget counters. The repository discloses all amendments and one accounting erratum.

References

- [1] E. Perez et al. FiLM: Visual reasoning with a general conditioning layer. *AAAI*, 2018.
- [2] W. Peebles, S. Xie. Scalable diffusion models with transformers. *ICCV*, 2023.
- [3] D. Ha, A. Dai, Q. Le. Hypernetworks. *ICLR*, 2017.
- [4] J. Su et al. RoFormer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864*, 2021.
- [5] Anonymous. Group representational position encoding. *ICLR*, 2026. *arXiv:2512.07805*.
- [6] E. Hu et al. LoRA. *ICLR*, 2022.
- [7] Z. Qiu et al. Orthogonal finetuning. *NeurIPS*, 2023.
- [8] W. Liu et al. BOFT. *ICLR*, 2024.
- [9] S. Yuan et al. Householder reflection adaptation. *NeurIPS*, 2024.
- [10] M. Gorbunov et al. Group and shuffle: Efficient structured orthogonal parametrization. *NeurIPS*, 2024.
- [11] R. Quessard, T. Barrett, W. Clements. Learning disentangled representations and group structure of dynamical environments. *NeurIPS*, 2020.
- [12] H. Caselles-Dupré, M. Garcia-Ortiz, D. Filliat. Symmetry-based disentangled representation learning requires interaction with environments. *NeurIPS*, 2019.